

ReQuSim: A simulation platform for quantum repeaters

Julius Wallnöfer^{1*}

¹ Dahlem Center for Complex Quantum Systems, Freie Universität Berlin,
Arnimallee 14, 14195 Berlin, Germany

* julius.wallnoefer@fu-berlin.de

August 23, 2024

Abstract

The abstract is in boldface, and should fit in 8 lines. It should be written in a clear and accessible style, emphasizing the context, the problem(s) studied, the methods used, the results obtained, the conclusions reached, and the outlook. You can add a table contents, recommended if your paper is more than 6 pages long.

Contents

1	Introduction	2
1.1	Installing ReQuSim	2
2	Working principles	2
2.1	Scope	3
3	User Guide	3
3.1	Elementary Structure	3
3.2	Building blocks for a simple repeater	5
3.3	Using the event system	6
3.3.1	A simple simulation loop	8
3.4	Noise and imperfections	9
3.4.1	Noise channels and custom noise models	9
3.4.2	Automatic application of time-dependent noises	11
3.4.3	Loss and initial noise	11
3.5	Interacting with the simulation	13
3.5.1	Obtaining information from the simulation	14
3.5.2	Using additional event information	14
3.5.3	Using the callback system	15
3.6	Advanced protocol options	15
3.6.1	Memory multiplexing	15
3.6.2	Entanglement Purification	16
3.6.3	Cutoff times	17
4	Benchmarking	17
5	Conclusion	17
A	Extending and customizing ReQuSim	17
A.1	Customizing events	17

A.2	Advanced customization of noise	18
A.2.1	Compatibility with delayed application	20
B	Examples	20
B.1	A simple repeater example	20
B.2	A full scale example	23
References		23

1 Introduction

Intro and Motivation here. Also mention other simulation packages here.

1.1 Installing ReQuSim

ReQuSim is available as a Python package on the Python package index for recent versions of Python 3 ($\geq 3.8.10$). You can install it via

```
pip install requsim
```

As with all python packages this can possibly overwrite already installed package versions in your environment with its dependencies (numpy $\geq 1.21.2$ and pandas $\geq 1.3.3$), which is why installing it in a dedicated virtual environment may be preferable.

For the specific version discussed in this document use **replace X.Y with actual version**

```
pip install requsim==X.Y
```

Source Code

The latest version of the source code is available on GitHub: [jwallnoefer/requisim](https://github.com/jwallnoefer/requisim). Version **vX.X** is also archived at DOI: **version specific zenodo doi link**. The documentation can be built from the source code, but the latest version is also hosted at: <https://requisim.readthedocs.io>.

2 Working principles

This whole paragraph needs reformulating... maybe start with explaining what I mean by state and changes to a state (i.e. quantum operations). At the core of the simulation is a list of events, that gets resolved in sequence. This *event queue* is what drives the simulation forward and causes the changes in the state of the simulation. If one talks about event systems in programming, it is often in the context of responding to user actions, e.g. pressing a key or interacting with a graphical user interface, however here the focus lies mostly on structuring changes to the state of the simulation in an appropriate order and making sure one event is fully resolved before triggering the next one. The ordering here is not performed according to any real time effects, as would be the case for e.g. requests to a server, but instead it makes sure events are resolved in the correct order in the simulated time.

Then go into the core mechanisms that make it run in reasonable time frame, e.g. only updating time-dependent effects when needed. an updated graphic maybe?

2.1 Scope

What can it do, what can't it do.

3 User Guide

This section describes basic concepts and how to use the ReQuSim from the ground up with the goal to enable the reader to both understand and customize all aspects of the simulation. For a quick start that skips the elementary aspects and delivers a straightforward simulation of a standard repeater protocol that works out of the box see the example in [Appendix so-and-so](#).

First, the core structure of a simulation is described, especially how certain parts of a quantum repeater protocol are represented in the code. While one does not necessarily need to interact with these low-level structures because more convenient tools for standard repeater scenarios are available, which hide some of this complexity, understanding these mechanisms should prove useful when coming up with custom scenarios or verifying that a protocol works as desired. Then, we will go over how these basic building blocks are combined to perform a simple repeater protocol. Following that, we discuss how noise and imperfections can be added to the simulation. Finally, multiple ways to interact with the simulation are described, which presents options for transforming a high-level description of a strategy into a ReQuSim-compatible protocol.

The documentation and API-Reference can be built from the source code using sphinx, but a copy is also hosted at requsim.readthedocs.io.

3.1 Elementary Structure

The basic structure needed for a ReQuSim simulation is a `World` object, which represents one particular repeater setup.

```
1 from requsim.world import World
2
3 world = World()
```

All quantum objects such as an entangled pair source, a repeater station or a qubit are represented as `WorldObject` instances. The `World` object is essentially a collection of all these objects belonging to one repeater setup (along with some automatic statistics). E.g. to initialize a `Qubit` object:

```
4 from requsim.quantum_objects import Qubit
5
6 qubit_1 = Qubit(world=world)
```

In the following we will briefly show how to interact with some of the low-level objects. While some standard protocols can be specified without this knowledge, the underlying structure is indispensable when defining custom behaviors.

Two qubits can be grouped into a `Pair` object, which is the main way ReQuSim represents bipartite entangled states, which are the core building block for a one-dimensional quantum repeater. Here, we create a Bell pair in the $|\Phi^+\rangle$ state.

```
7 qubit_2 = Qubit(world=world)
8
9 from requsim.quantum_objects import Pair
10 import requsim.libs.matrix as mat
11
12 density_matrix_phiplus = mat.phiplus @ mat.H(mat.phiplus)
```

```

13 pair = Pair(world=world,
14             qubits=[qubit_1, qubit_2],
15             initial_state=density_matrix_phiplus)

```

`requsim.libs.matrix` is a subpackage that contains common quantum states and operators as numpy arrays, as well as common quantum operations and maps as functions acting on numpy arrays. `mat.H` is a function that returns the complex conjugate of a numpy array.

The `World` object keeps track of all objects belonging to the setup. `World.world_objects` is a dictionary that contains all world objects that are active in the simulation, separated by their `WorldObject.type` property, e.g. `world.world_objects["Pair"]` is a list containing all `Pair` objects (if following the code snippets so far, there should be one pair). Furthermore, by calling `world.print_status()` one can print an overview of the objects managed by the `World` object, which can be useful for debugging.

The quantum state ρ of the pair object can be accessed directly by `pair.state` and can simply be updated by setting it to a new state. For example, the application of a Pauli-X operator on the first qubit can be done by:

```

16 x_on_first_qubit = mat.tensor(mat.X, mat.I(2))
17 pair.state = x_on_first_qubit @ pair.state @ mat.H(x_on_first_qubit)

```

To remove quantum objects from the simulation, either because they are no longer present (e.g. after a destructive measurement operations) or because they are no longer interesting to the considered protocol, the `WorldObject.destroy` method can be used. In the ongoing example, one could remove all quantum objects like this:

```

18 pair.destroy()
19 qubit_1.destroy()
20 qubit_2.destroy()

```

Note that removing a `Pair` object in this fashion does not automatically destroy the constituent qubits, since they might be reused and become part of another pair (e.g. after entanglement swapping). However, if a qubit is removed, the higher order `Pair` will also be removed, since its state is no longer properly defined without the existence of all qubits.

This direct manipulation of states can in principle be used to represent arbitrary operations on such pair states, such as entanglement swapping:

```

21 qubit_a = Qubit(world=world)
22 qubit_b1 = Qubit(world=world)
23 pair_1 = Pair(world=world,
24              qubits=[qubit_a, qubit_b1],
25              initial_state=density_matrix_phiplus)
26 qubit_b2 = Qubit(world=world)
27 qubit_c = Qubit(world=world)
28 pair_2 = Pair(world=world,
29              qubits=[qubit_b2, qubit_c],
30              initial_state=density_matrix_phiplus)
31
32 # entanglement swapping by projecting qubits b1 and b2 on the phi_plus state
33 four_qubit_state = mat.tensor(pair_1.state, pair_2.state)
34 operator = mat.tensor(mat.I(2), mat.H(mat.phiplus), mat.I(2))
35 state_after_swapping = operator @ four_qubit_state @ mat.H(operator)
36 state_after_swapping = state_after_swapping / np.trace(state_after_swapping)
37 new_pair = Pair(world=world,
38               qubits=[qubit_a, qubit_c],
39               initial_state=state_after_swapping)
40
41 # remove outdated objects
42 pair_1.destroy()
43 pair_2.destroy()
44 qubit_b1.destroy()

```

```
45 | qubit_b2.destroy()
```

3.2 Building blocks for a simple repeater

While the manual manipulation of qubits and pairs is a good starting point, the utility of a simulation platform comes from automating those aspects that one encounters regularly. In order to facilitate this, ReQuSim expects things to be organized according to certain structures.

Station objects represent the repeater stations and end stations of a quantum repeater. They can hold qubits, which effectively represents them having a quantum memory, although the specifics of the memory are not described explicitly.

```
1 | from requsim.world import World
2 | from requsim.quantum_objects import Station
3 |
4 | world = World()
5 | station_a = Station(world=world, position=0)
6 | station_b = Station(world=world, position=2000) # meters
```

The `position` argument can be a number (for linear repeater chains) or a list or numpy array of coordinates. A qubit that is associated with a station, (i.e. in physical terms stored in a quantum memory at that station) can be instantiated with the `create_qubit` method:

```
7 | qubit_a = station_a.create_qubit()
8 | qubit_b = station_b.create_qubit()
9 | print(qubit_a in station_a.qubits) # True
10 | print(qubit_b in station_a.qubits) # False
```

This association of station and qubit is often desirable if some features of the setup are tied directly to where the qubit is stored. This will be especially useful when considering noise and imperfections, such as a time-dependent memory noise.

Another central element of repeater setups is the distribution of elementary entangled states. Entangled pair sources are represented by Source objects, that distribute pair-states between their target stations.

```
11 | from requsim.quantum_objects import Source
12 |
13 | source = Source(world=world,
14 |                position=1000,
15 |                target_stations=[station_a, station_b])
```

The `Source.generate_pair` method allows to create an arbitrary pair state with a qubit at each of the target stations. It is important to note that this places the state in the simulation immediately, not accounting for travel times (see the `SchedulingSource` below for a convenient way to do so). For example, creating $|\Phi^+\rangle$ state can be achieved by:

```
16 | import requsim.libs.matrix as mat
17 |
18 | source.generate_pair(initial_state=mat.phiplus @ mat.H(mat.phiplus))
```

One can easily verify the state of the simulation now includes a pair between the two stations:

```
>>> world.print_status()
Event queue:
%=====
Objects
-----
Station:
Station 1
Station 2
-----
```

Qubit:

Qubit 1 at station Station 1, part of None.
 Qubit 2 at station Station 2, part of None.
 Qubit 3 at station Station 1, part of Pair 1.
 Qubit 4 at station Station 2, part of Pair 1.

Source:

Source 1 generating states between stations Station 1, Station 2.

Pair:

Pair 1 with qubits Qubit 3, Qubit 4 between stations Station 1, Station 2.

This also shows the two qubits, which were generated before.

3.3 Using the event system

While the manual manipulation of quantum objects in this framework is clearly possible, the added value over direct matrix operations comes from embedding them in a more formalized way of transforming the state of the simulation. This is done using an event system and each event represents either an action that interacts with quantum states (like applying a quantum operation) or otherwise changes the elements of the simulation (like making a resource available after an appropriate amount of time has passed for the necessary classical information to be transmitted). Unlike the event systems that more commonly deal with reacting to real-time changes, e.g. in asynchronous applications or interactive graphical user interfaces, this event system concerns itself with the simulated time passing in the simulation instead. However, in addition to simply providing an order to the changes that happen in the simulation, concepts such as callbacks or event listeners can also be useful in this case for interacting with the simulation. **is there any good textbook source for callbacks/event systems that I can cite here? there must be, right?**

The event queue is represented by an EventQueue object, and each repeater setup comes with its own event queue via `World.event_queue`. The event queue is essentially a list of all scheduled events `EventQueue.queue` and methods to schedule new events (`add_event`) as well as to move the simulation forward via `resolve_next_event`.

Each type of change of simulation needs its own event type and we will introduce some of the built-in events as they come up. A generic event type `GenericEvent` is also available, which simply calls a function with a set of pre-defined arguments and could, in principle, be used to make arbitrary changes to the simulation. However, subclassing `Event` is the recommended way to add new functionality (see Appendix A for more detailed instructions).

Let us consider a setup with two parties connected by a single repeater station that also controls the entangled pair sources and introduce some common type of events in that context.

```

1 from requsim.world import World
2 from requsim.quantum_objects import Station, Source
3 C = 2e8 # speed of light in optical fiber
4
5 world = World()
6 station_a = Station(world=world, position=0, label="Station A")
7 station_central = Station(world=world, position=1000, label="Station central")
8 station_b = Station(world=world, position=2000, label="Station B")
9
10 source_a = Source(world=world,
11                   position=station_central.position,
12                   target_stations=[station_a, station_central])
13 source_b = Source(world=world,
14                   position=station_central.position,
15                   target_stations=[station_central, station_b])

```

Now schedule the generation of an entangled pair per link, taking into account the delay of sending one qubit of each pair to the end stations:

```

16 from requisim.libs.aux_functions import distance
17 import requisim.libs.matrix as mat
18 from requisim.events import SourceEvent
19
20 density_matrix_phiplus = mat.phiplus @ mat.H(mat.phiplus)
21 delay_a = distance(source_a, station_a) / C
22 delay_b = distance(source_b, station_b) / C
23 event_a = SourceEvent(time=world.event_queue.current_time + delay_a,
24                       source=source_a,
25                       initial_state=density_matrix_phiplus)
26 world.event_queue.add_event(event_a)
27 event_b = SourceEvent(time=world.event_queue.current_time + delay_b,
28                       source=source_b,
29                       initial_state=density_matrix_phiplus)
30 world.event_queue.add_event(event_b)

```

SourceEvent is the event type to distribute an entangled state⁴ for an elementary link. We can check that the two events are now scheduled:

```

>>> world.print_status()
Event queue:
SourceEvent at time=5e-06 generating a state between stations Station A, Station
central.
SourceEvent at time=5e-06 generating a state between stations Station central,
Station B.
=====
Objects
-----
Station:
Station A
Station central
Station B
-----
Source:
Source 1 generating states between stations Station A, Station central.
Source 2 generating states between stations Station central, Station B.

```

Next, let all events resolve:

```

16 while world.event_queue.queue:
17     world.event_queue.resolve_next_event()

```

Then schedule the entanglement swapping with an EntanglementSwappingEvent. Since only two pairs exist, identifying the correct ones is easy in this case - in more complex cases additional checks or the use of the information returned by the resolution of the source events would be necessary.

```

18 from requisim.events import EntanglementSwappingEvent
19 pairs_to_swap = [world.world_objects["Pair"][0], world.world_objects["Pair"][1]]
20 swap_event = EntanglementSwappingEvent(time=world.event_queue.current_time,
21                                       pairs=pairs_to_swap,
22                                       station=station_central)
23 world.event_queue.add_event(swap_event)
24 world.event_queue.resolve_next_event()

```

A look at the current state of the simulation reveals that, as expected, a long distance pair has been established:

```

>>> world.print_status()
Event queue:
=====

```

Objects

Station:
 Station A
 Station central
 Station B

Source:
 Source 1 generating states between stations Station A, Station central.
 Source 2 generating states between stations Station central, Station B.

Qubit:
 Qubit 1 at station Station A, part of Pair 3.
 Qubit 4 at station Station B, part of Pair 3.

Pair:
 Pair 3 with qubits Qubit 1, Qubit 4 between stations Station A, Station B.

As a technical detail, `EntanglementSwappingEvent` assumes that the outcome of the Bell state measurement was $|\Phi^+\rangle$. However, in practice the result of the measurement must be communicated to A and B. Depending on the application, A and B may need to wait for this information before continuing to use the entangled state or may be able to account for the necessary corrections in post-processing (which is the case e.g. for Clifford operations and Pauli measurements). For now, we just verify, that the distributed state is indeed a Bell pair and remove the successfully established pair from the simulation:

```
21 long_distance_pair = world.world_objects["Pair"][0]
22 import numpy as np
23 assert np.allclose(long_distance_pair.state, density_matrix_phiplus) # True
24 long_distance_pair.destroy()
25 for qubit in long_distance_pair.qubits:
26     qubit.destroy()
```

3.3.1 A simple simulation loop

In order to run a continuous simulation, one can set up the procedure described above as an automated protocol. In principle, one can use any mechanism to schedule events as desired, but here we will use one of the included `Protocol` classes provided by `requisim.tools` as a demonstration (continuing the the scenario already established in the previous section).

```
27 from requisim.tools.protocol import TwoLinkProtocol
28
29 class ExampleProtocol(TwoLinkProtocol):
30     def check(message=None):
31         left_pairs = self._get_left_pairs()
32         num_left_pairs = len(left_pairs)
33         num_left_pairs_scheduled = len(self._left_pairs_scheduled())
34         right_pairs = self._get_right_pairs()
35         num_right_pairs = len(right_pairs)
36         num_right_pairs_scheduled = len(self._right_pairs_scheduled())
37         long_distance_pairs = self._get_long_range_pairs()
38
39         # If there are no pairs established and no pairs scheduled: Schedule a pair.
40         if num_left_pairs + num_left_pairs_scheduled == 0:
41             event_a = SourceEvent(time=world.event_queue.current_time + delay_a,
42                                   source=source_a,
43                                   initial_state=density_matrix_phiplus)
44             world.event_queue.add_event(event_a)
45         if num_right_pairs + num_right_pairs_scheduled == 0:
46             event_b = SourceEvent(time=world.event_queue.current_time + delay_b,
```

```

47         source=source_b,
48         initial_state=density_matrix_phiplus)
49     world.event_queue.add_event(event_b)
50
51     # If both links are present, do entanglement swapping.
52     if num_left_pairs == 1 and num_right_pairs == 1:
53         left_pair = left_pairs[0]
54         right_pair = right_pairs[0]
55         ent_swap_event = EntanglementSwappingEvent(
56             time=self.world.event_queue.current_time,
57             pairs=[left_pair, right_pair],
58             station=self.station_central
59         )
60         self.world.event_queue.add_event(ent_swap_event)
61
62     # If a long range pair is present, save its data and delete it.
63     if long_distance_pairs:
64         for pair in long_distance_pairs:
65             self._eval_pair(pair)
66             for qubit in pair.qubits:
67                 qubit.destroy()
68             pair.destroy()

```

The `TwoLinkProtocol` provides multiple of helper methods that filter the objects in the `world` to obtain the relevant ones. One can then instantiate and set up the the protocol interact with the simulation, by calling the `check` method to repeatedly schedule events. E.g. to run the simulation until 100 long-distance pairs are distributed one can do the following:

```

69 protocol = ExampleProtocol()
70 protocol.setup(world=world, communication_speed=C)
71
72 num_target_pairs = 100
73 current_message = None
74 while len(protocol.time_list) < num_target_pairs:
75     protocol.check(message=current_message)
76     current_message = world.event_queue.resolve_next_event()

```

With a main loop that calls the `protocol.check` after every resolved event, the protocol gets to react to every event and can schedule new ones continuously. The output data, a collection of all the times when they were distributed and the states of all the distributed pairs (in the form of a `pandas.DataFrame`) can then be obtained via:

```

77 protocol.data

```

3.4 Noise and imperfections

Any realistic protocol will invariably be confronted with noise and imperfections. In fact, taking these into account is a central challenge for developing and analyzing repeater protocols. ReQuSim offers ways to apply arbitrary noise maps as part of other operations and take loss into account. First, we will show how arbitrary noise maps can be represented in the noise framework and how to manually use them to update quantum states. Then, we will discuss a few ways how ReQuSim can take care of some types of noise automatically by specifying the relevant noise channels and noise models as parameters for quantum objects and events.

3.4.1 Noise channels and custom noise models

Noise maps are represented by `NoiseChannel` objects. The map is specified as a Python function that takes a n -qubit state as input and returns a state of the same size as output.

This means custom error models can be specified by providing the appropriate function for the corresponding CPTP map.

For example, a single-qubit phase flip error with an error parameter ϵ can be specified as follows:

```

1 import requsim.libs.matrix as mat
2 from requsim.noise import NoiseChannel
3
4 def phase_flip_func(rho, epsilon):
5     return (1 - epsilon) * rho + epsilon * mat.Z @ rho @ mat.Z
6
7 phase_flip_channel = NoiseChannel(n_qubits=1, channel_function=phase_flip_func)

```

The channel can then be applied to any state, e.g. to apply it to the first qubit of a pair state:

```

8 from requsim.world import World
9 from requsim.quantum_objects import Qubit, Pair
10
11 density_matrix_phiplus = mat.phiplus @ mat.H(mat.phiplus)
12 pair = Pair(world=world,
13             qubits=[Qubit(world), Qubit(world)],
14             initial_state=density_matrix_phiplus)
15
16 pair.state = phase_flip_channel.apply_to(rho=pair.state,
17                                         qubit_indices=[0],
18                                         epsilon=0.1)

```

The above code changes the state of the entangled pair according to:

$$\rho' = (1 - \epsilon)\rho + \epsilon(Z \otimes \mathbb{1})\rho(Z \otimes \mathbb{1}) \quad (1)$$

with $\epsilon = 0.1$ as an error parameter of the channel.

Often when discussing what happens to a quantum state, the effect is given not as a map acting on the whole system, but on parts of it, e.g. *The first qubit is affected by a phase-flip channel*. This way of specifying and applying noise maps via `NoiseChannel` explicitly allows applying such locally defined error models to be applied to states of arbitrary size.

Special consideration is given to single-qubit noise channels, as acting on a specific qubit is fairly common, e.g. the memory noise may depend only on where the qubit is stored. For convenience, single qubit noise can directly be applied from the qubit object without having to figure out the `qubit_indices` required by the `apply_to` method. The following has the same effect as the last instruction in the code above.

```

19 pair.qubits[0].apply_noise(phase_flip_channel, epsilon=0.1)

```

Some common noise channels, such as single-qubit Pauli noise and fully depolarizing noise are provided with ReQuSim via the `requsim.tools.noise_channels` module. `requsim.tools` is a subpackage that is not directly necessary for running simulations, but contains helpful definitions that come up regularly when describing quantum repeaters.

Does this need a multi-qubit example?

As a side note, since arbitrary callables can be specified as the channel function, it is possible to include error models that vary over time with relatively low effort (see [Appendix something something](#) for an example).

Some of the built-in events also support custom `NoiseModel` objects, which allow to specify separate noise channels that act before and after the main operation (which is assumed perfect), or, alternatively replace the standard operation with a different map. Refer to the documentation of the individual event types to see the possibilities and the exact requirements.

3.4.2 Automatic application of time-dependent noises

One central source of imperfection that comes up regularly for quantum repeaters are the imperfect memories. The Station object supports adding a time-dependent noise channel that takes care of these effects, e.g. for a dephasing noise channel with a dephasing time T_{dp} it works like this:

```

1 from requisim.tools.noise_channels import z_noise_channel
2 from requisim.noise import NoiseChannel
3 from requisim.world import World
4 from requisim.quantum_objects import Station
5 import numpy as np
6 T_DP = 100e-3 # 100 ms
7 def phase_flip(rho, t): # t is a time interval
8     error_parameter = (1 - np.exp(-t / T_DP)) / 2
9     return z_noise_channel(rho=rho, epsilon=error_parameter)
10
11 dephasing_noise_channel = NoiseChannel(n_qubits=1, channel_function=phase_flip)
12
13 world = World()
14 noisy_station = Station(world=world,
15                         position=4000,
16                         memory_noise=dephasing_noise_channel)

```

The `z_noise_channel` is a single-qubit phase flip channel that is equivalent to the example channel of Section 3.4.1. Now all qubits at that station will be affected by this time-dependent noise automatically whenever their state is updated.

The parameter `t` that is used for this call is the time that has passed since the last update.

3.4.3 Loss and initial noise

Another essential aspect of any repeater scheme is how the elementary entangled states that connect neighbouring stations are established. One central concern in this phase is the loss of qubits while in transmission, which is very significant, e.g. for photons in an optical fibre. In principle, one could simulate every single qubit that is sent, however, if the success probability is very low, this comes with a significant overhead.

The main intended way for ReQuSim to handle this is to instead draw from a probability distribution that specifies when the next successful attempt to establish an entangled connection will happen. A convenient way to do so is provided via the `SchedulingSource` subclass of `Source`.

One can provide two functions that will be called whenever a successful distribution of an elementary entangled state should be scheduled as an event. One is the `time_distribution` that should return the time interval until the next successful attempt, usually drawn from a probability distribution. The other is the `state_generation`, which should return the state of the distributed state at the point in time when success has been confirmed.

The following is an example for a fixed success probability per trial, each of which takes a fixed trial time and generate perfect initial Bell states.

```

1 from requisim.world import World
2 import numpy as np
3 import requisim.libs.matrix as mat
4 from requisim.quantum_objects import Station, SchedulingSource
5
6 T_TRIAL = 100e-3 # 100 ms
7 P_SUC = 0.2 # success probability
8 density_matrix_phiplus = mat.phiplus @ mat.H(mat.phiplus)
9
10 def fixed_initial_state_generation(source):

```

```

11 |     return density_matrix_phiplus
12 |
13 | def flat_probability_time_distribution(source):
14 |     num_trials = np.random.geometric(P_SUC)
15 |     return num_trials * T_TRIAL
16 |
17 | world = World()
18 | station_a = Station(world=world, position=0)
19 | station_b = Station(world=world, position=1000)
20 | source = SchedulingSource(world=world, position=500,
21 |                           target_stations=[station_a, station_b],
22 |                           time_distribution=flat_probability_time_distribution,
23 |                           state_generation=fixed_initial_state_generation)

```

Then, an event can be generated and scheduled automatically:

```

25 | source.schedule_event()

```

This calls the `flat_probability_time_distribution` and draws randomly from a geometric distribution to find the time of the next successful trial.

A variety of different processes can be modelled by assigning different `time_distribution` and `state_generation` function. For example, one could add an initial noise that is applied to every pair as part of an imperfect generation process (which could also include errors from, e.g., loading the generated qubits into a quantum memory). For depolarizing noise, this could be written as:

```

26 | F_INIT = 0.99
27 |
28 | def depolarizing_initial_state_generation(source):
29 |     noisy_state = F_INIT * (mat.phiplus @ mat.H(mat.phiplus)) + (1 - F_INIT) / 3 *
30 |         (
31 |             mat.psiplus @ mat.H(mat.psiplus)
32 |             + mat.phiminus @ mat.H(mat.phiminus)
33 |             + mat.psiminus @ mat.H(mat.psiminus)
34 |         )
35 |     return noisy_state

```

These functions take the assigned source object as argument in order to make it straightforward to include distance dependent effects dynamically. This becomes relevant when considering common features such as distance-dependent loss and possibly the need to confirm successful establishment of an entangled pair via classical communication. The following is a demonstration of how to implement a common setup.

Consider two stations A and B are separated by a distance l and connected via a quantum channel. Station A also houses an entangled pair source. The probability that a qubit that is a transmitted through the channel arrives at the other end is distance dependent: $\eta(l) = e^{-l/L_{\text{att}}}$ with an attenuation length L_{att} . A trial consists of A generating an entangled pair, storing one qubit in their quantum memory and sending the other through the lossy channel to B , which takes a time interval l/c where c is the communication speed (e.g. 2×10^8 m/s for optical fibres at telecom wavelengths). Then, B signals back whether the qubit arrived at their station (Note: this implies a heralded process for loading the qubit into memory). Only after another time l/c station A will know whether the trial has been successful. So in total, one trial would take $t_{\text{trial}} = 2l/c$. This also means when the pair is confirmed to be established, the qubit at station A has already been stored (and therefore affected by time-dependent memory noise) for

$$t_A = 2\frac{l}{c} \quad (2)$$

and the qubit at station B has already been stored for

$$t_B = \frac{l}{c} \quad (3)$$

This can be captured by the following functions: **check imports again or maybe this should be a fully separat example**

```

35 c = 2e8 # m/s. Speed of light in optical fibre.
36 L_ATT = 22000 # m. Also optical fibre.
37
38 def time_distribution(source): # this assumes the source is positioned at one of
    the stations
39     comm_distance = np.max([distance(source, station) for station in source.
        target_stations])
40     trial_time = 2 * comm_distance / c
41     eta = np.exp(-comm_distance/L_ATT)
42     random_number_of_trials = np.random.geometric(eta)
43     return random_number_of_trials * trial_time
44
45 def state_generation(source):
46     state = density_matrix_phiplus
47     comm_distance = np.max([distance(source, station) for station in source.
        target_stations])
48     trial_time = 2 * comm_distance / c
49     for idx, station in enumerate(source.target_stations):
50         if station.memory_noise is not None:
51             # while qubit and classical information are travelling dephasing already
occurs
52             # the amount of time depends on where the station is located
53             storage_time = trial_time - distance(source, station) / c
54             state = apply_single_qubit_map(
55                 map_func=station.memory_noise,
56                 qubit_index=idx,
57                 rho=state,
58                 t=storage_time,
59             )
60     return state

```

3.5 Interacting with the simulation

ReQuSim does not inherently restrict what events can be scheduled or what information can be used to do so and provides multiple ways to obtain information about what is currently happening in the simulation. There are three main ways to react to changes in the state of the simulation, which have some built-in considerations:

- Use `world.event_queue` and `world.world_objects` to analyze the complete state of the simulation and determine the next step. This allows to access all the information the simulation has, but comes with performance considerations.
- When events are resolved, either by manually calling the `Event.resolve` or, more commonly, by the `EventQueue.resolve_next_event` method, they will return a message in the form of the dictionary, that contains information about what was done by the event.
- When events are scheduled, one can add arbitrary callables as callbacks to events or event types that will be called with the message returned by the event after it has resolved.

These methods can also be mixed and matched as appropriate for the specific repeater protocol one wishes to consider. In the following a few examples and considerations for each of these methods are discussed.

3.5.1 Obtaining information from the simulation

As already mentioned in Section 3.1, the `World` object collects and registers all created entities in the simulation sorted by their type.

One can use this to find, e.g. all `Pair`s that are shared between two stations, which we will call `station_A` and `station_B` in the following example:

```
def filter_func(pair):
    return pair.is_between_stations(station_A, station_B)

pairs_AB = list(filter(filter_func, world.world_objects["Pair"]))
```

Alternatively, one could find all entangled pair sources, which distribute states between `station_A` and `station_B`:

```
def filter_func(source):
    return (station_A in source.target_stations
            and station_B in source.target_stations)

sources_AB = list(filter(filter_func, world.world_objects["Source"]))
```

One could use a similar process, to find all qubits at a certain station, however, in that case just accessing `Station.qubits` will likely be preferable.

It can be desirable to look at which events are already scheduled, e.g. to avoid scheduling the same event twice. One could find all events which will establish a pair between `station_A` and `station_B` in the future, i.e. in physical terms, the associated memories are currently busy with trying and retrying the generation process:

```
def filter_func(event):
    return (isinstance(event, SourceEvent)
            and (self.station_A in event.source.target_stations)
            and (self.station_B in event.source.target_stations))

source_events_AB = list(filter(filter_func, world.event_queue.queue))
```

This is exactly the way the example protocol in Section [the simple loop example](#) is obtaining the number of scheduled pairs.

The obvious drawback of this method is, that the lists of this *global perspective* that need to be filtered can grow quite large for larger setups (see also Section 4). As an alternative, the protocol could keep a record of the events when it schedules them that is already structured by the repeater link, so not the whole event queue needs to be consulted each time.

3.5.2 Using additional event information

Each event provides some additional information when it is resolved. The information provided can vary (see the API Reference in the documentation for details), but will always contain at least the event type and whether the event resolved successfully. A resolution of the event is considered unsuccessful, e.g. if the required objects do no longer exist in the simulation at the time of resolution.

The type of the event can be useful for the protocol to gauge what could be the next required step, e.g. if the last event was a `SourceEvent` the protocol can expect that there is now a `Pair` object in the simulation that was not there when it last checked.

Consider, that as part of the protocol there is a function `check_at_station` that should be called whenever a new elementary entangled pair is established between two stations:

```
def example_protocol_func(message):
    if message["event_type"] == "SourceEvent" and message["resolve_successful"]:
        affected_stations = message["source"].target_stations
        for station in affected_stations:
```

```

        check_at_station(station)
    # other protocol logic goes here

while True: # main loop
    message = world.event_queue.resolve_next_event()
    example_protocol_func(message=message)

```

Consider, as another example, a repeater chain with many stations. For the purpose of detecting when a pair between the stations at the end of the chain (`station_A` and `station_B`) has been established and therefore should be recorded, it could be useful to check the long distance output pair after each entanglement swapping rather than having to filter through all `Pair` objects at each step:

```

def example_protocol_func(message):
    if (message["event_type"] == "EntanglementSwappingEvent"
        and message["resolve_successful"]):
        output_pair = message["output_pair"]
        if output_pair.is_between_stations(station_A, station_B):
            on_end_stations_reached(output_pair)
    # other protocol logic goes here

```

3.5.3 Using the callback system

It is also possible to add callbacks to individual events, which will be automatically called after the event has been resolved. Consider, as an example, the **TODO: continue here**

3.6 Advanced protocol options

On the conceptional side of quantum communication, one direction of research of particular interest is to explore and analyze advanced strategies and situations. While the core idea the repeater protocol is well established, when it comes to optimizing protocols outside of standard scenarios, especially when considering asymmetries, much less is known. One appeal of approaching these settings as numerical simulations is that the modular nature makes combining multiple approaches relatively straightforward. **How much in depth should the dicussion go here? Maybe check again after introduction is written?**

In the following we discuss how ReQuSim supports some commonly used elements for repeater protocols.

3.6.1 Memory multiplexing

One can support multiple memories at each repeater station by simply scheduling more events that will create pairs. It is the responsibility of the user to define appropriate probability distributions and schedule the correct amount events so the generation process captures correctly how many memories are kept busy by successive attempts to establish the initial pairs.

For example, a straightforward way to model m memories on each side of the elementary link, is to simply call the `schedule_event` method introduced above in Section 3.4.3 m times.

Do note, however, that ReQuSim does not come with any advanced memory management, i.e. there is no tracking which qubit occupies which slot exactly. If something like that is required for a use case, this would need to be built on top of the existing systems. However, since the qubits in `station.qubits` are added in order of generation, it is easy to tell in which order the corresponding pairs were established. This is useful when formulating protocols in terms of, e.g. using *the most recent pair* for entanglement swapping.

3.6.2 Entanglement Purification

Entanglement purification protocols (EPP) (e.g. [1–3]) are a class of protocols that can be used to deal with noise one inevitably encounters. Generally, they take multiple copies of a noisy entangled state and, by using only local operations and classical communication, transform them - potentially probabilistically - into fewer copies of the same state with less noise. The use of entanglement purification is an important tool for entanglement-swapping-based repeaters and has been a consideration from the very beginning [4,5].

ReQuSim provides a built-in event type that can be used to implement $m \rightarrow 1$ protocols for bipartite entangled states.

Consider the following example of applying one step of the DEJMPS entanglement purification protocol [2] on two input pairs:

```

1 from requsim.world import World
2 from requsim.quantum_objects import Pair, Station
3 import requsim.libs.matrix as mat
4 from requsim.libs.aux_functions import distance
5
6 C = 2e8 # speed of light in optical fiber
7 F_INIT = 0.8
8 noisy_initial_state = (F_INIT * (mat.phiplus @ mat.H(mat.phiplus)) +
9                        (1 - F_INIT) / 3 * (mat.psiplus @ mat.H(mat.psiplus)
10                                           + mat.phiminus @ mat.H(mat.phiminus)
11                                           + mat.psiminus @ mat.H(mat.psiminus)))
12
13 world = World()
14 station_a = Station(world=world, position=0)
15 station_b = Station(world=world, position=2000)
16 pair_1 = Pair(world=world, qubits=[station_a.create_qubit(),
17                                   station_b.create_qubit()],
18               initial_state=noisy_initial_state)
19 pair_2 = Pair(world=world, qubits=[station_a.create_qubit(),
20                                   station_b.create_qubit()],
21               initial_state=noisy_initial_state)
22
23 epp_event = EntanglementPurificationEvent(time=0, pairs=[pair_1, pair_2],
24                                           communication_time=distance(station_a, station_b)/C,
25                                           protocol="dejmps")
26 world.event_queue.add_event(epp_event)
27 world.event_queue.resolve_next_event()

```

A crucial step of this protocol is that the measurement outcomes of the qubits on the second pair need to be known to both parties performing the EPP in order to assess whether the entanglement purification step was successful. In ReQuSim this exchange of the classical information is not modeled in detail, but the incurred delay is taken into account via the `communication_time` parameter.

While waiting for this information the remaining entangled pair should usually not be used for anything else. This is signified by the event setting the `blocked` attribute of the output pair to `True`. Events that try to act on blocked objects usually do nothing and print a warning, unless they are explicitly set to be allowed to so. In principle, one could speculatively use the output pair, before it is known whether the entanglement purification was successful.

After the communication time has passed, the pair will be unblocked by an `UnblockEvent`, if the entanglement purification has been successful, or discarded via `DiscardQubitEvents`, otherwise. These events are scheduled when the entanglement `EntanglementPurificationEvent` resolves, since the outcome is already known by the simulation, even if the parties cannot know yet.

The `EntanglementPurificationEvent` can use a pre-defined entanglement purifica-

tion protocol (as of ReQuSim 0.5 this is only the DEJMPS protocol), or any bipartite $n \rightarrow 1$ by passing a callable that takes a tensor product of pair states as input and returns the success probability and the state of the output pair in case the protocol was successful (see the documentation for details).

3.6.3 Cutoff times

4 Benchmarking

I guess this should be its own section, I guess?

5 Conclusion

We conclude... that there is a code now. Maybe use this as outlook.

Acknowledgements

The author thanks Frederik Hahn and Fabian Wiesner for helpful discussions and their willingness to be test users for an earlier, unreleased version of ReQuSim. Their feedback was invaluable for improving the usability of the code. Furthermore, the author thanks Johannes Jakob Meyer for interesting discussions on structuring code for representing quantum objects.

Funding information The author acknowledges support from the Federal Ministry of Education and Research (BMBF) via projects Q.Link.X and QR.X.

A Extending and customizing ReQuSim

A.1 Customizing events

Adding new event types or modifying existing ones should be done by subclassing the `Event` class.

A subclass of `Event` needs to define a `_main_effect` method and an appropriate `__repr__`; adding a easily interpretable `__str__` is also encouraged, as this will show up e.g. in `world.print_status()`. What the event itself does when it resolves should be defined in the `_main_effect` method. It should be called without any arguments and return a dictionary or `None`. The keys and values of the dictionary will be part of what will be returned by calling the `resolve` method, so this can be used to include additional information about what the event did. Overriding the `resolve` method itself (which does some additional checks and bookkeeping), should usually not be required.

In the following we will elaborate on some details of the internals of ReQuSim, which one usually does not encounter when just using the built-in events, but could be relevant when extending it.

When interacting or changing quantum states, it is important to make sure all time-dependent effects have been applied to the state up to the point in time the event happens. This means calling the `update_time` method on the relevant objects. Updating the time on a `Pair` object, also automatically updates it on all constituent qubits. All built-in events take care of this already, but custom events will make sure to do so manually.

Furthermore, it is important to make sure that the new event defines sensible `required_objects`, as the event will fail to resolve if those are no longer present in the simulation, i.e. the event will instead do nothing and set `"resolve_successful"` to `False` (callbacks will still happen).

Consider as an example a new kind of event that does not perform any operation but simply fetches the current state of a pair at a certain time. A minimal code for this purpose would like this:

```

1 from requesim.events import Event
2
3 class UpdatePairEvent(Event):
4     def __init__(self, time, pair, callback_functions=None):
5         self.pair = pair
6         super().__init__(time=time,
7                           required_objects=[pair],
8                           callback_functions=callback_functions)
9
10    def __repr__(self):
11        return (self.__class__.__name__
12                + f"(time={self.time}, pair={self.pair}, "
13                + f"callback_functions={self._callback_functions})")
14
15    def _main_effect(self):
16        self.pair.update_time()
17        return {"pair": self.pair, "state": self.pair.state}

```

If modifying existing event types, consider whether updating the `type` property of the event to use that of the superclass instead may be desirable, in order to remain compatible with existing scheduling logic that looks for the name of the event.

A.2 Advanced customization of noise

Section 3.4 already introduces how arbitrary maps can be used to define custom noise channels.

However, since the `NoiseChannel` accepts any callable, this offers remarkable flexibility in what can be modelled. In the following examples, this is used to define noise effects that vary over time, but other applications are conceivable (like external triggers that turn on or off noise processes).

Let us consider single-qubit dephasing noise channels that have a time-dependent dephasing probability:

$$\mathcal{D}_z(t)\rho = p(t)\rho + \frac{1-p(t)}{2}[\rho + Z\rho Z], \quad (4)$$

i. e. $p(t)$ is the probability that the phase information of the state is retained.

Often the dephasing is treated as a constate rate $\lambda = 1/T_{\text{dp}}$, which is the inverse of a dephasing time T_{dp} , and p is then given by:

$$dp = -\lambda p dt \quad (5)$$

Together with starting condition $p(t=0) = 1$ one obtains:

$$p(t) = e^{-\lambda t} \quad (6)$$

This error channel has the convenient effect that (for $t_2 > t_1$) it holds that

$$\mathcal{D}_z(t_2 - t_1) \circ \mathcal{D}_z(t_1) = \mathcal{D}_z(t_2), \quad (7)$$

because $p(t_2) = p(t_2 - t_1)p(t_1)$ and therefore the total channel can be composed from multiple time intervals. This common type of channel is the reason why the built-in handling of memory

noise (by defining them when initializing the associated `Station` object) relies completely on calling the associated noise channel with the time interval since the last update.

Now, however, let us consider a dephasing rate that varies over time. Let us consider a global fluctuation of the quality of the memory:

$$dp = -\lambda(1 + \alpha \sin \frac{t}{\tau})p dt \quad (8)$$

with some parameters $0 < \alpha \leq 1$ and $\tau > 0$. This leads to the expression:

$$p(t) = C e^{-\lambda t} e^{\lambda \alpha \tau \cos(t/\tau)} \quad (9)$$

where C is a constant that depends on at the boundary condition, i.e. if the qubit was loaded into the quantum memory at time t_0 one has $p(t = t_0) = 1$. we write this the following way:

$$p_{t_0}(t) = e^{-\lambda(t-t_0)} e^{\lambda \alpha \tau [\cos(t/\tau) - \cos(t_0/\tau)]} \quad (10)$$

Conveniently this can also be decomposed to allow for incrementally applying the noise whenever the state is updated:

$$p_{t_0}(t_2) = p_{t_0}(t_1) * p_{t_1}(t_2) \quad (11)$$

In terms of code, this means that by providing the noise access to the global simulation time, one can calculate the appropriate $p_{t_1}(t_2)$ from just the time interval since the last update, which the function will be called with automatically.

Therefore we can define a new callable object that implements the above channel like this:

TODO: Test that this actually works.

```

1 import requisim.libs.matrix as mat
2 import numpy as np
3 from requisim.noise import NoiseChannel
4 from requisim.world import World
5
6 def custom_dephasing_function(rho, p): # as defined in text above
7     return p * rho + (1-p) / 2 * (rho + mat.Z @ rho @ mat.H(mat.Z))
8
9
10 class FluctuatingDephasingNoise(object):
11     def __init__(self, world, rate, alpha, tau):
12         self.world = world
13         self.rate = rate
14         self.alpha = alpha
15         self.tau = tau
16
17     def __call__(self, rho, t):
18         # t is the time interval since last update
19         current_time = self.world.event_queue.current_time
20         last_update_time = current_time - t
21         p = np.exp(-self.rate * t) * (np.exp(self.rate * self.alpha * self.tau *
22                                             (np.cos(current_time / self.tau) -
23                                              np.cos(last_update_time / self.tau)))
24                                     )
25         return custom_dephasing_function(rho, p)
26
27 world = World()
28 fluctuating_noise = FluctuatingDephasingNoise(world, 1/0.1, alpha=0.5, tau=1)
29 fluctuating_dephasing_channel = NoiseChannel(n_qubits=1,
30                                              channel_function=fluctuating_noise)

```

The channel defined as such can then be used in places where a noise channel is expected, e.g. as the time-dependent memory noise of a station:

```

31 from requsim.quantum_objects import Station
32
33 station = Station(world=world, position=0,
34                  memory_noise=fluctuating_dephasing_channel)

```

More options for noise channels More complex models for noise channels can be done in this way by defining classes with appropriate `__call__` methods. For example, one could instantiate a new callable object for every qubit and progress their noise effects individually. However, these would have to be assigned to the qubits manually, as the built in `memory_noise` parameter does not support that.

If a detailed model or simulation of the physical system represented by the abstract qubit objects is available, defining a custom noise channel like above could be a viable way to make it compatible with ReQuSim.

Compatibility with delayed application of Channel As a sidenote, the `NoiseChannel` class has a `freeze` method that allows to store an application of the channel for later. This is sometimes necessary due to internal details of ReQuSim that allow `Qubit` objects to exist without being assigned to any higher order object that handles the application of noise and tracking of the state (like `Pair`).

If a `Pair` is always assigned in the same time step, as is the case for all scenarios discussed in this guide and in the examples, no further action is required for the custom noise channel above. However, if, for some reason, this cannot be guaranteed, the referencing of the current time in the simulation as done in the example above, will not produce the correct noise channels. A customised `freeze` method would be necessary to handle these circumstances.

B Examples

Here are two examples that show how ReQuSim can be used in practice. More examples are available with the source code (in the `docs/examples/` directory).

Furthermore, the codebases for the scenarios used in [6] and [7] are archived at [8] and [9], respectively. These show how `requsim` has been used in practice to answer research questions. However, do be aware that these use an outdated version of ReQuSim, so not everything may be perfectly compatible.

B.1 A simple repeater example

Check for accuracy with newest version before submission.

```

1 import numpy as np
2 import pandas as pd
3 from requsim.tools.protocol import TwoLinkProtocol
4 from requsim.tools.noise_channels import z_noise_channel
5 from requsim.tools.evaluation import standard_bipartite_evaluation
6 from requsim.libs.aux_functions import distance
7 import requsim.libs.matrix as mat
8 from requsim.events import EntanglementSwappingEvent
9 from requsim.world import World
10 from requsim.noise import NoiseChannel, NoiseModel
11 from requsim.quantum_objects import Station, SchedulingSource
12

```

```

13
14 def construct_dephasing_noise_channel(dephasing_time):
15     def lambda_dp(t):
16         return (1 - np.exp(-t / dephasing_time)) / 2
17
18     def dephasing_noise_func(rho, t):
19         return z_noise_channel(rho=rho, epsilon=lambda_dp(t))
20
21     return NoiseChannel(n_qubits=1, channel_function=dephasing_noise_func)
22
23
24 class SimpleProtocol(TwoLinkProtocol):
25     # TwoLinkProtocol is a helpful abstract class that provides useful
26     # attributes and methods for dealing with and evaluating repeater protocols
27     # consisting of two links (supports multi-mode memories as well).
28     # It needs to be initialized via its 'setup' method after world setup
29     # is complete, in order to find all the relevant objects in the world.
30     def check(self, message=None):
31         """Check current status and schedule new events.
32
33         This looks globally at the status of the whole 'world' and decides
34         which events should be scheduled because of it. It is intended to be
35         called after every change in the 'world'.
36
37         Parameters
38         -----
39         message : dict or None
40             Optional additional information for the Protocol to consider.
41             This particular protocol does not use it. Default is None.
42
43         Returns
44         -----
45         None
46
47         """
48         left_pairs = self._get_left_pairs()
49         num_left_pairs = len(left_pairs)
50         num_left_pairs_scheduled = len(self._left_pairs_scheduled())
51         right_pairs = self._get_right_pairs()
52         num_right_pairs = len(right_pairs)
53         num_right_pairs_scheduled = len(self._right_pairs_scheduled())
54         long_distance_pairs = self._get_long_range_pairs()
55
56         # STEP 1: For each link, if there are no pairs established and
57         # no pairs scheduled: Schedule a pair.
58         if num_left_pairs + num_left_pairs_scheduled == 0:
59             self.source_A.schedule_event()
60         if num_right_pairs + num_right_pairs_scheduled == 0:
61             self.source_B.schedule_event()
62
63         # STEP 2: If both links are present, do entanglement swapping.
64         if num_left_pairs == 1 and num_right_pairs == 1:
65             left_pair = left_pairs[0]
66             right_pair = right_pairs[0]
67             ent_swap_event = EntanglementSwappingEvent(
68                 time=self.world.event_queue.current_time,
69                 pairs=[left_pair, right_pair],
70                 station=self.station_central,
71             )
72             self.world.event_queue.add_event(ent_swap_event)
73
74         # STEP 3: If a long range pair is present, save its data and delete

```

```

75         # the associated objects.
76         if long_distance_pairs:
77             for pair in long_distance_pairs:
78                 self._eval_pair(pair)
79                 for qubit in pair.qubits:
80                     qubit.destroy()
81                 pair.destroy()
82
83
84 def run(length, max_iter, params):
85     C = params["COMMUNICATION_SPEED"]
86     P_LINK = params["P_LINK"]
87     T_DP = params["T_DP"]
88     LAMBDA_BSM = params["LAMBDA_BSM"]
89     L_ATT = 22e3 # attenuation length
90
91     # define functions for link generation behavior
92     def state_generation(source):
93         # this returns the density matrix of a successful trial
94         # this particular function already assumes some things that are only
95         # appropriate for this particular setup, e.g. the source is at one
96         # of the stations and the end stations do not decohere
97         state = mat.phiplus @ mat.H(mat.phiplus)
98         comm_distance = max(
99             distance(source, source.target_stations[0]),
100             distance(source, source.target_stations[1]),
101         )
102         storage_time = 2 * comm_distance / C
103         for idx, station in enumerate(source.target_stations):
104             if station.memory_noise is not None:
105                 state = station.memory_noise.apply_to(
106                     rho=state, qubit_indices=[idx], t=storage_time
107                 )
108         return state
109
110     def time_distribution(source):
111         comm_distance = max(
112             distance(source, source.target_stations[0]),
113             distance(source, source.target_stations[1]),
114         )
115         trial_time = 2 * comm_distance / C
116         eta = P_LINK * np.exp(-comm_distance / L_ATT)
117         num_trials = np.random.geometric(eta)
118         time_taken = num_trials * trial_time
119         return time_taken
120
121     def BSM_error_func(rho):
122         return LAMBDA_BSM * rho + (1 - LAMBDA_BSM) * mat.I(4) / 4
123
124     BSM_noise_channel = NoiseChannel(n_qubits=2, channel_function=BSM_error_func)
125     BSM_noise_model = NoiseModel(channel_before=BSM_noise_channel)
126
127     # perform the world setup
128     world = World()
129     station_A = Station(world=world, position=0)
130     station_central = Station(
131         world=world,
132         position=length / 2,
133         memory_noise=construct_dephasing_noise_channel(T_DP),
134         BSM_noise_model=BSM_noise_model,
135     )
136     station_B = Station(world=world, position=length)

```

```

137 source_A = SchedulingSource(
138     world=world,
139     position=station_central.position,
140     target_stations=[station_A, station_central],
141     time_distribution=time_distribution,
142     state_generation=state_generation,
143 )
144 source_B = SchedulingSource(
145     world=world,
146     position=station_central.position,
147     target_stations=[station_central, station_B],
148     time_distribution=time_distribution,
149     state_generation=state_generation,
150 )
151 protocol = SimpleProtocol()
152 protocol.setup(world=world, communication_speed=C)
153
154 current_message = None
155 while len(protocol.time_list) < max_iter:
156     protocol.check(message=current_message)
157     current_message = world.event_queue.resolve_next_event()
158 return protocol
159
160
161 if __name__ == "__main__":
162     params = {
163         "P_LINK": 0.80, # link generation probability
164         "T_DP": 100e-3, # dephasing time
165         "LAMBDA_BSM": 0.99, # Bell-State-Measurement ideality parameter
166         "COMMUNICATION_SPEED": 2e8, # speed of light in optical fibre
167     }
168     length_list = np.linspace(20e3, 200e3, num=8)
169     max_iter = 1000
170     raw_data = [
171         run(length=length, max_iter=max_iter, params=params).data
172         for length in length_list
173     ]
174     result_list = [standard_bipartite_evaluation(data_frame=df) for df in raw_data]
175     results = pd.DataFrame(
176         data=result_list,
177         index=length_list,
178         columns=[
179             "raw_rate",
180             "fidelity",
181             "fidelity_std_err",
182             "key_per_time",
183             "key_per_time_std_err",
184         ],
185     )
186     print(results)
187     # plotting key_per_time vs. length is usually what you want to do with these

```

docs/examples/event_protocol_simple_repeater.py

B.2 A full scale example

References

- [1] C. H. Bennett, G. Brassard, S. Popescu, B. Schumacher, J. A. Smolin and W. K. Wootters, *Purification of noisy entanglement and faithful teleportation via noisy channels*, Phys. Rev. Lett. **76**, 722 (1996), doi:[10.1103/PhysRevLett.76.722](https://doi.org/10.1103/PhysRevLett.76.722).
- [2] D. Deutsch, A. Ekert, R. Jozsa, C. Macchiavello, S. Popescu and A. Sanpera, *Quantum privacy amplification and the security of quantum cryptography over noisy channels*, Phys. Rev. Lett. **77**, 2818 (1996), doi:[10.1103/PhysRevLett.77.2818](https://doi.org/10.1103/PhysRevLett.77.2818).
- [3] W. Dür and H. J. Briegel, *Entanglement purification and quantum error correction*, Reports on Progress in Physics **70**(8), 1381 (2007), doi:[10.1088/0034-4885/70/8/R03](https://doi.org/10.1088/0034-4885/70/8/R03).
- [4] H.-J. Briegel, W. Dür, J. I. Cirac and P. Zoller, *Quantum repeaters: The role of imperfect local operations in quantum communication*, Phys. Rev. Lett. **81**, 5932 (1998), doi:[10.1103/PhysRevLett.81.5932](https://doi.org/10.1103/PhysRevLett.81.5932).
- [5] W. Dür, H.-J. Briegel, J. I. Cirac and P. Zoller, *Quantum repeaters based on entanglement purification*, Phys. Rev. A **59**, 169 (1999), doi:[10.1103/PhysRevA.59.169](https://doi.org/10.1103/PhysRevA.59.169).
- [6] J. Wallnöfer, F. Hahn, M. Gündoğan, J. S. Sidhu, F. Wiesner, N. Walk, J. Eisert and J. Wolters, *Simulating quantum repeater strategies for multiple satellites*, Communications Physics **5**(1), 169 (2022), doi:[10.1038/s42005-022-00945-9](https://doi.org/10.1038/s42005-022-00945-9).
- [7] J. Wallnöfer, F. Hahn, F. Wiesner, N. Walk and J. Eisert, *Faithfully simulating near-term quantum repeaters*, PRX Quantum **5**, 010351 (2024), doi:[10.1103/PRXQuantum.5.010351](https://doi.org/10.1103/PRXQuantum.5.010351).
- [8] J. Wallnöfer, F. Hahn and F. Wiesner, *Code archive for "Simulating quantum repeater strategies for multiple satellites"*, doi:[10.5281/zenodo.6384558](https://doi.org/10.5281/zenodo.6384558) (2022).
- [9] J. Wallnöfer, F. Hahn and F. Wiesner, *Code archive for "Faithfully simulating near-term quantum repeaters"*, doi:[10.5281/zenodo.10083186](https://doi.org/10.5281/zenodo.10083186) (2023).